Similarly, both or and || return their first argument unless it is falsey, in which case they evaluate and return their second argument.

```
nil || 99   # => 99
false || 99 # => 99
"cat" || 99 # => "cat"
```

As with and, the only difference between or and || is their precedence. To make life interesting, and and or have the same precedence, but && has a higher precedence than ||.

The spelled out versions of and and or are useful as a kind of control flow, EXPRESSION or exit will perform the expression, no matter how complex it is, and return that value if it is truthy, exiting only if the entire expression is false.

A common idiom is to use ||= to assign a value to a variable only if that variable isn't already set:

```
var ||= "default value"
```

This is almost, but not quite, the same as var = var || "default value". It differs in that no assignment is made at all if the variable is already set. In pseudocode, this might be written as var = "default value" unless var or as var || var = "default value".

not and ! (logical not) return the opposite of their operand (false if the operand is any true value and true if the operand is any false value). And, yes, not and ! differ only in precedence. You'll sometimes see a !! used as an implicit conversion to boolean, since the first ! converts any value to either true or false and the second ! reverses the value to match the boolean status of the original value.

The defined? Keyword

The defined? keyword returns nil if its argument (which can be an arbitrary expression) is not defined in the current scope; otherwise, it returns a description of that argument. If the argument is yield, defined? returns the string "yield" if a code block is associated with the current context.

```
defined? 1        # => "expression"
defined? dummy    # => nil
defined? printf   # => "method"
defined? String   # => "constant"
defined? $_       # => "global-variable"
defined? Math::PI # => "constant"
defined? a = 1    # => "assignment"
defined? 42.abs   # => "method"
defined? nil      # => "nil"
```

Comparing Objects

In addition to the boolean operators, Ruby objects support comparison using the methods ==, ===, <=>, =~, eql?, and equal? (see Table 3, Common comparison operators, on page 152). All but <=> are defined in class Object but are often overridden by descendants to provide appropriate semantics. For example, class Array redefines == so that two array objects are equal if they have the same number of elements and the corresponding elements are equal.

It's relatively rare to see eql? or equal? in Ruby code. Also, if you are familiar with JavaScript, please note that triple-equal, === means something very different in Ruby than in JavaScript.